Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria

En Ingeniería Y Tecnologías Avanzadas.

Arquitecturas para bases de datos distribuidas y gestores de bases de datos distribuidas

Basado en el libro de M. Tamer Özsu y Patrick Valduriez
Principles of Distributed Database Systems (3rd ed., Springer, 2011)
y en documentación técnica oficial consultada en Internet

Alumno:

Mendoza Xicohtencatl Hector Jair

Profesor: De la Cruz Sosa Carlos

Grupo: 3TM2

Fecha de entrega: 20/03/2026

Introducción

Una base de datos distribuida busca que los usuarios trabajen con una sola base lógica, aun cuando los datos se encuentren almacenados en varios sitios o nodos. A partir de esa base teórica, este reporte aborda dos temas. Primero, las arquitecturas para bases de datos distribuidas, retomando los modelos de referencia expuestos por el libro. Segundo, una revisión comparativa de varios gestores y plataformas actuales para observar qué estrategias de distribución soportan, especialmente fragmentación y replicación.

Tema 1. Arquitecturas para bases de datos distribuidas

En el libro base, la arquitectura de un sistema se entiende como la definición de sus componentes, la función de cada uno y la forma en que interactúan. Para un SGBD distribuido, esta arquitectura debe explicar cómo fluye el control, dónde residen los datos y cómo se coordinan los nodos participantes.

El texto presenta tres arquitecturas de referencia para un DBMS distribuido: cliente/servidor, peer-to-peer y sistemas multidatabase. Aunque son modelos idealizados, siguen siendo útiles para clasificar soluciones reales y entender cómo resuelven problemas como autonomía local, heterogeneidad, transparencia de distribución y coordinación de consultas.

1.1 Arquitectura cliente/servidor

En este enfoque existe una separación funcional entre los clientes, que generan solicitudes, y los servidores, que almacenan datos y ejecutan parte importante del procesamiento. Su ventaja principal es la claridad organizacional: la administración y el control suelen concentrarse del lado del servidor, mientras el cliente conserva una vista relativamente simple del sistema.

Esta arquitectura fue muy importante en los primeros sistemas distribuidos comerciales y sigue vigente en muchas plataformas que poseen un nodo coordinador o catálogo central. Su desventaja aparece cuando ese componente central se convierte en cuello de botella para coordinación, enrutamiento o consultas globales.

1.2 Arquitectura peer-to-peer (P2P)

En la arquitectura peer-to-peer no existe una distinción rígida entre cliente y servidor. Cada nodo puede almacenar datos, procesar consultas y cooperar con otros nodos. Esto incrementa la escalabilidad y reduce la dependencia de un único punto central, por lo que resulta atractiva para entornos grandes y cambiantes.

Sin embargo, la coordinación de transacciones, el mantenimiento de consistencia y la localización eficiente de datos suelen ser más complejos.

1.3 Sistemas multidatabase o federados

La tercera arquitectura de referencia agrupa sistemas multidatabase o federados. Aquí la prioridad no es únicamente distribuir datos de una sola base, sino integrar múltiples bases autónomas, posiblemente heterogéneas, para ofrecer acceso conjunto. Esto es útil cuando las organizaciones ya poseen varios sistemas locales y no desean perder autonomía administrativa.

Su fortaleza es la integración de fuentes distintas; su reto más evidente es la heterogeneidad: distintos modelos de datos, distintos esquemas, diferentes dialectos de consulta y diversas políticas de control. Por ello, esta arquitectura suele priorizar interoperabilidad y mediación más que uniformidad total.

1.4 Relación entre arquitectura y diseño de distribución

El libro distingue también el diseño de distribución de datos. Dentro de este diseño, la fragmentación puede ser horizontal, vertical o híbrida. La fragmentación horizontal divide una relación por tuplas o filas; la fragmentación vertical divide por atributos o columnas; y la híbrida combina ambas. Además, la asignación puede incluir replicación para mejorar disponibilidad, tolerancia a fallos y rendimiento de lectura.

En términos prácticos, la arquitectura elegida condiciona el modo en que se implementan estos mecanismos. Un sistema cliente/servidor puede centralizar metadatos y decisiones; un sistema P2P suele repartirlas; y un sistema federado añade el problema extra de traducir y coordinar entre motores distintos.

Tema 2. Gestores de bases de datos distribuidas

Para este segundo apartado se revisaron gestores y plataformas ampliamente documentados en línea. El criterio de comparación fue identificar qué tanto soportan la fragmentación y la replicación.

2.1 Comparativo general

Gestor / plataforma	Fragmentación / distribución	Replicación	Soporte vertical
Oracle Sharding	Sí, horizontal nativa. Tablas sharded y particiones distribuidas por shards; además maneja tablas duplicadas.	Sí. Data Guard y, en versiones recientes, también opciones como Raft para disponibilidad.	No se documenta como función nativa general.
Citus (PostgreSQL)	Sí, horizontal. Usa distribution column y shards; también reference tables.	Sí. Réplicas de shards y streaming replication; factor configurable.	No nativa general.
CockroachDB	Sí, particionamiento por filas y geo-partitioning.	Sí. Réplicas entre nodos con consenso y factor configurable.	No nativa general.
YugabyteDB	Sí, horizontal por hash o rango; tablets/shards automáticos.	Sí. Replicación síncrona con Raft; también xCluster asíncrono entre universos.	No nativa general.
MySQL NDB Cluster	Sí. Particionado automático y también KEY / LINEAR KEY en tablas NDB.	Sí. Réplicas de fragmentos dentro del clúster y replicación asíncrona entre clusters.	No nativa general.
PostgreSQL (núcleo)	Parcial. Particionamiento declarativo y distribución lógica por tablas particionadas, pero no DDBMS completo multi-nodo por sí solo.	Sí. Replicación física y lógica, incluso para tablas particionadas.	No nativa general.

2.2 Análisis por gestor

Oracle Sharding

Oracle describe una base sharded como una sola base lógica, horizontalmente particionada a través de varios shards físicos. Además, incorpora tablas duplicadas que se replican en todos los shards para disminuir consultas multi-shard. En replicación, ofrece alta disponibilidad mediante Data Guard y, en documentación reciente, opciones de replicación basadas en Raft. Por ello, Oracle presenta una de las implementaciones más completas de la combinación fragmentación horizontal + replicación.

Citus sobre PostgreSQL

Citus transforma PostgreSQL en una base de datos distribuida. Sus tablas distribuidas usan una columna de distribución para asignar filas a shards, mientras que las reference tables se replican en todos los workers. También admite factor de replicación para shards. Esto lo vuelve adecuado para cargas multi-tenant y para escalar SQL horizontalmente sin abandonar el ecosistema PostgreSQL.

CockroachDB

CockroachDB integra distribución y replicación desde el diseño del motor. Permite particionamiento por filas y geo-partitioning, pero la característica más distintiva es que la replicación se apoya en consenso y quórum, con factor configurable. En otras palabras, la distribución no se maneja como complemento, sino como parte esencial de la arquitectura del sistema.

YugabyteDB

YugabyteDB soporta sharding por hash y por rango, con tablets que pueden dividirse y redistribuirse automáticamente. Por defecto replica datos de manera síncrona con Raft para consistencia fuerte, y además dispone de xCluster para replicación asíncrona entre universos. Esto le da mucha flexibilidad para combinar escalamiento horizontal, resiliencia y despliegue geográficamente distribuido.

MySQL NDB Cluster

NDB Cluster particiona automáticamente las tablas NDB y permite ciertos esquemas definidos por el usuario, especialmente KEY y LINEAR KEY. Internamente maneja fragment replicas dentro del cluster; adicionalmente puede replicar de un cluster a otro mediante replicación asíncrona. Comparado con otros sistemas recientes, sus capacidades de distribución son robustas, aunque más específicas y con restricciones claras.

PostgreSQL sin extensiones

PostgreSQL, por sí solo, no se presenta como un DDBMS completo multi-nodo. No obstante, sí aporta dos piezas fundamentales: particionamiento declarativo y replicación lógica/física. En la práctica, esto lo convierte en una base excelente para soluciones distribuidas, especialmente cuando se amplía con extensiones como Citus.

Referencias consultadas

Özsu, M. T., & Valduriez, P. (2011). Principles of Distributed Database Systems (3rd ed.). Springer.

Springer Nature. Principles of Distributed Database Systems. Página del libro y tabla de contenido.

Oracle Documentation. Oracle Sharding Architecture and Concepts; Oracle Globally Distributed AI Database Overview.

PostgreSQL Documentation. Table Partitioning; Logical Replication; Logical Replication Restrictions.

Citus Documentation. Concepts; Citus Tables and Views; Configuration Reference.

CockroachDB Docs. Table Partitioning; Replication Layer.

YugabyteDB Docs. DocDB sharding layer; Hash and range sharding; Replication in DocDB; xCluster replication.

MySQL Reference Manual. NDB Cluster Nodes, Node Groups, Fragment Replicas, and Partitions; NDB Cluster Replication.